

Laporan Tugas Akhir
“Analisis Clustering dan Klasifikasi untuk Deteksi Kanker
Payudara Menggunakan K-Means dan Decision Tree”



Disusun Oleh Kelompok 08 2024 A:

Aida Zahira	(24031554106)
Fauziah Roikhana Wardah	(24031554162)
Juli Yawati Manalu	(24031554050)

Dosen Pengampu:

Yuni Rosita Dewi, M.Si.

PROGRAM STUDI S1 SAINS DATA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA
TAHUN AJARAN 2025/2026

DAFTAR ISI

KATA PENGANTAR.....	3
BAB I.....	4
PENDAHULUAN.....	4
A. Latar Belakang.....	4
B. Tujuan.....	5
C. Manfaat.....	5
BAB II.....	6
DATASET.....	6
A. Sumber Data.....	6
B. Exploratory Data Analysis.....	6
BAB III.....	10
METODE.....	10
A. Penentuan Metode.....	10
B. Kesesuaian Metode dengan Penyelesaian Masalah dalam Project.....	10
C. Penentuan Metrik Evaluasi.....	11
BAB IV.....	13
HASIL DAN ANALISIS.....	13
A. Hasil Eksperimen atau Pemodelan.....	13
B. Analisis atau Penjelasan Hasil Eksperimen.....	27
C. Interactive Search.....	29
D. Diagram Alur.....	31
BAB V	
KESIMPULAN DAN SARAN.....	32
A. Kesimpulan.....	32
B. Kendala Berdasarkan Hasil Eksperimen.....	32
C. Rencana Lanjut dari Hasil Eksperimen.....	33
DAFTAR PUSTAKA.....	33
LAMPIRAN.....	34

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karuniaNya, kami dapat menyelesaikan laporan tugas akhir di semester 3 dengan baik. Laporan yang berjudul “Analisis Clustering dan Klasifikasi untuk Deteksi Kanker Payudara Menggunakan K-Means dan Decision Tree” ini disusun sebagai tugas UAS untuk menyelesaikan mata kuliah Kecerdasan Artifisial prodi Sains Data di Universitas Negeri Surabaya.

Kami menyadari bahwa penyusunan laporan ini atas kerja sama kelompok. Oleh karena itu, sebelumnya kami ingin mengucapkan terima kasih kepada ibu Yuni Rosita Dewi, M.Si. selaku dosen mata kuliah Kecerdasan Artifisial yang telah memberikan kami tugas ini untuk melatih kemampuan kami dalam menulis laporan. Dan tak lupa kami ucapkan terima kasih pada diri kami yang telah rela meluangkan waktunya demi menjalankan tanggung jawab sebagai mahasiswa dalam menjalankan tugasnya.

Kami berharap dengan menulis laporan ini dapat memberikan kontribusi yang menarik dan berkesan bagi pembaca. Kami juga menyadari bahwa laporan ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang kalian sampaikan sangat berarti bagi kami untuk memperbaiki di masa yang akan datang. Akhir kata, semoga laporan ini dapat bermanfaat bagi semua pembaca.

Surabaya, 15 Desember 2025

BAB I

PENDAHULUAN

A. Latar Belakang

Kanker payudara merupakan salah satu masalah kesehatan utama dan penyakit mematikan yang paling ditakuti, terutama di kalangan wanita di seluruh dunia[1]. Tingginya penderita penyakit ini di Indonesia diperkirakan mencapai jutaan, dimana mengharuskan adanya upaya deteksi dan diagnosis yang cepat serta akurat. Kanker payudara umumnya dikelompokkan menjadi dua jenis ganas dan jinak, dan penentuan jenis ini memerlukan metode yang cepat untuk membantu pengambilan keputusan medis[1]. Untuk meningkatkan kemungkinan keberhasilan pengobatan dan secara signifikan mengurangi angka kematian, deteksi dini kanker payudara yang akurat sangatlah penting[2].

Metode diagnosis tradisional seringkali bergantung pada analisis citra medis, seperti MRI membutuhkan keahlian tinggi dari dokter spesialis, yang mana prosesnya bisa memakan waktu dan berpotensi mengandung bias subjektif. Oleh karena itu, bidang *Machine Learning* sebagai bagian dari kecerdasan buatan menawarkan solusi yang menjanjikan dalam membantu diagnosis medis. *Machine Learning* berfokus pada penerapan algoritma khusus prediksi, pengenalan pola, dan klasifikasi data, sehingga dapat digunakan untuk mengklasifikasikan jenis kanker payudara secara otomatis dan cepat[1].

Tugas akhir ini mengintegrasikan dua teknik *Machine Learning* yang berbeda, yaitu *K-Means* untuk proses analisis data awal dan *Decision Tree* untuk proses klasifikasi akhir. Algoritma *K-Means* digunakan sebagai teknik *clustering unsupervised* yang sangat relevan untuk memproses data medis. Dalam konteks analisis citra, *K-Means* dapat digunakan untuk segmentasi atau pengelompokan piksel citra MRI guna memperjelas objek kanker, sehingga area tumor dapat diidentifikasi dan dikelompokkan dengan lebih mudah dan akurat[3].

Dan ada algoritma *Decision Tree*, teknik klasifikasi yang populer karena kemampuannya menghasilkan model transparan dan mudah dipahami untuk mendukung pengambilan keputusan medis[4]. Dalam studi sebelumnya, algoritma *Decision Tree* telah menunjukkan potensi besar dalam mengklasifikasikan data kanker payudara dengan tingkat akurasi yang tinggi, yakni mencapai 95.00% dalam pengelompokan kanker jinak dan ganas[1].

Dengan menggabungkan analisis clustering menggunakan algoritma *K-Means* untuk pemrosesan data awal dan klasifikasi menggunakan Algoritma *Decision Tree*, tugas akhir ini bertujuan untuk membangun sebuah sistem deteksi kanker payudara yang lebih efisien, terukur dan memberikan interpretasi yang jelas mengenai faktor-faktor yang paling berpengaruh dalam diagnosis.

B. Tujuan

1. Membuat sebuah sistem deteksi kanker payudara yang mampu mengklasifikasikan jenis kanker termasuk jinak atau ganas secara cepat dan akurat dengan memanfaatkan kombinasi algoritma *K-Means* dan *Decision Tree*.
2. Membangun model klasifikasi berbasis *Decision Tree* yang dapat memberikan hasil yang mudah untuk dipahami.
3. Menyediakan sistem yang dapat mempercepat proses identifikasi kanker payudara dan tidak hanya bergantung pada analisis manual saja.
4. Mengidentifikasi fitur yang paling berpengaruh dalam membedakan tumor jinak dan ganas melalui struktur *Decision Tree*.

C. Manfaat

1. Menyediakan alat bantu klasifikasi otomatis yang dapat membantu mempercepat proses identifikasi awal kanker payudara.
2. Meningkatkan potensi deteksi dini, dengan memberi sinyal awal yang membantu dokter mengambil tindakan lebih cepat.
3. Memberikan informasi tentang fitur yang paling berpengaruh dalam diagnosis sebagai dukungan tambahan bagi tenaga medis.

BAB II DATASET

A. Sumber Data

Dataset yang digunakan dalam project ini dari Kaggle. Kaggle merupakan platform dataset yang banyak digunakan sebagai tempat pengambilan data. Kaggle juga digunakan sebagai tempat menyediakan kumpulan data yang telah di akurasi dan dapat diunduh secara gratis sehingga memudahkan proses eksplorasi serta analisis data.

Dataset yang diambil dalam project ini adalah *Breast Cancer Dataset*. Dataset ini merupakan kumpulan data medis yang digunakan untuk deteksi kanker payudara. Terdapat 30 fitur numerik berkelanjutan seperti rata-rata, standar *error*, dan nilai *worst* dari karakteristik sel. Dataset umum yang digunakan untuk klasifikasi yaitu membedakan tumor jinak dan tumor ganas.

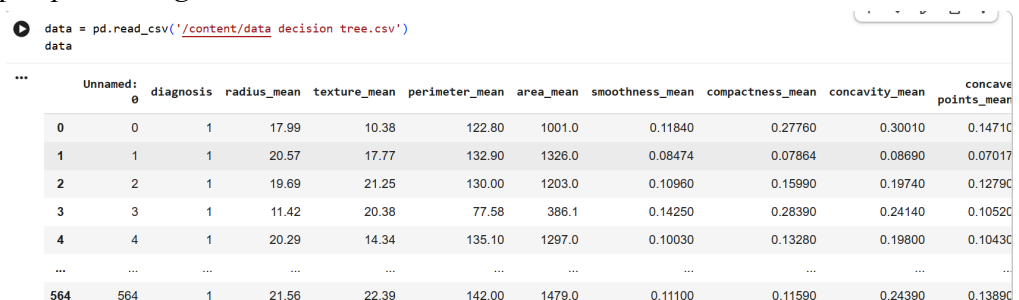
Data yang diperoleh dari Kaggle disediakan dalam format CSV sehingga bisa langsung digunakan untuk pemrograman. Dataset yang digunakan untuk melakukan klasifikasi kanker payudara berdasarkan fitur yang ada. Oleh karena itu, dataset kaggle ini menjadi dasar untuk mendeteksi jenis tumor yang akurat dan efisien.

Link Kaggle : <https://www.kaggle.com/datasets/wasiqaliyasir/breast-cancer-dataset>

B. Exploratory Data Analysis

Exploratory Data Analysis merupakan langkah penting karena memvisualisasi data untuk memahami fitur utamanya, menemukan pola, dan bagaimana bagian data terhubung. *Exploratory Data Analysis* untuk memahami data dalam memilih teknik pemodelan dan penyesuaian hasil yang baik[5]. Dalam tahap EDA dilakukan dua kali EDA untuk masing-masing metode. Tahap EDA untuk *Decision Tree* terdapat 6 tahap yaitu:

1. Load data untuk menampilkan apakah benar data *Decision Tree* yang sudah di *pre-processing*.



```
data = pd.read_csv('/content/data decision tree.csv')
data
```

...	Unnamed: 0	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	0	1	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710
1	1	1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017
2	2	1	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790
3	3	1	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520
4	4	1	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430
...	...	...	...	...	...	...	...	...	...	...
564	564	1	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890

Gambar 1. Load Data

2. Terdapat kolom *unnamed* yang tidak diperlukan sehingga dalam tahap ini diperlukan untuk menghapus kolom *unnamed*.

```
drop_cols = [col for col in data.columns if "Unnamed" in col]
data = data.drop(columns=drop_cols, errors="ignore")
data
```

...

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smc
0	1	17.99	10.38	122.80	1001.0	
1	1	20.57	17.77	132.90	1326.0	
2	1	19.69	21.25	130.00	1203.0	
3	1	11.42	20.38	77.58	386.1	
4	1	20.29	14.34	135.10	1297.0	
...	...	...	...	...	...	...
564	1	21.56	22.39	142.00	1479.0	
565	1	20.13	28.25	131.20	1261.0	
566	1	16.60	28.08	108.30	858.1	

Gambar 2. Menghapus Kolom Unnamed

3. Dilakukan `data.info()` untuk mengecek info terkait kolom tersebut apakah *not null* dan *type* dalam kolom masing-masing.

```
data.info()
```

...

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   diagnosis                             569 non-null    int64
1   radius_mean                           569 non-null    float64
2   texture_mean                           569 non-null    float64
3   perimeter_mean                         569 non-null    float64
4   area_mean                             569 non-null    float64
5   smoothness_mean                       569 non-null    float64
6   compactness_mean                       569 non-null    float64
7   concavity_mean                         569 non-null    float64
8   concave points_mean                   569 non-null    float64
9   symmetry_mean                         569 non-null    float64
10  fractal_dimension_mean                 569 non-null    float64
11  radius_se                              569 non-null    float64
12  texture_se                             569 non-null    float64
13  perimeter_se                           569 non-null    float64
14  area_se                                569 non-null    float64
```

Gambar 3. Cek Info

- Melakukan pengecekan seberapa banyak data jumlah baris dan kolom yang ada di dalam file menggunakan `data.shape`.

```
data.shape
```

```
... (569, 31)
```

Gambar 4. Menghitung Data

- Melakukan describe data untuk menampilkan statistika deskriptif.

```
data.describe()
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	sym
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	0.372583	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.048919
std	0.483918	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.038803
min	0.000000	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.000000
25%	0.000000	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.020310
50%	0.000000	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.033500
75%	1.000000	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.074000
max	1.000000	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.201200

8 rows x 31 columns

Gambar 5. Statistika Deskriptif

- Melakukan pengecekan ulang terhadap data apakah masih ada data missing value atau tidak.

```
missing = data.isnull().sum()
if missing.sum() > 0:
    print("Missing values:")
    print(missing[missing > 0])
else:
    print("Tidak ada missing values")
```

```
... Tidak ada missing values
```

Gambar 6. Missing Values

Namun, dalam tahap EDA *K-Means* terdapat 5 tahap yaitu:

- Load data untuk membaca dan menampilkan data seperti *gambar 1*.
- Melakukan `data.info()` untuk melihat tipe data fitur dan *non null*nya dapat dilihat dalam *gambar 2*.
- Melakukan `data.shape` untuk melihat jumlah baris dan kolom seperti *gambar 3*.
- Menampilkan statistika deskriptif untuk melihat nilai maksimum, minimum, dan rata-rata menggunakan `data.describe()` yang dilakukan dalam *gambar 4*.
- Melakukan pengecekan missing values pada data apakah sudah tidak ada atau masih ada dalam *gambar 5*.

BAB III METODE

A. Penentuan Metode

Pada penentuan metode untuk project yang dilakukan yaitu menggunakan algoritma *Decision Tree* dan *K-Means*. Pemilihan kedua metode ini dilakukan untuk klasifikasi jenis tumor dan mengelompokkan berdasarkan sel tumor.

1. *Decision Tree*

Decision Tree digunakan untuk membentuk pohon keputusan yang terdiri dari node dengan membentuk root, node tidak memiliki input. Node lainnya dinamakan daun sebagai nilai target yang paling tepat dari salah satu class[1]. *Decision tree* digunakan untuk klasifikasi diagnosis tumor dalam bentuk jenisnya. Pemilihan metode ini dilakukan karena mudah dipahami dan diinterpretasikan, mampu menangani data numerik, dan akan memberikan prediksi kelas tumor serta mengidentifikasi fitur yang paling menentukan hasil klasifikasi.

2. *K-Means*

K-Means merupakan algoritma clustering yang bersifat iteratif untuk mempartisi dataset menjadi subkelompok *non-overlapping* yang ditentukan oleh K(cluster) dengan setiap titik data dimiliki satu kelompok[6]. *K-Means* digunakan untuk mengelompokkan data secara unsupervised tanpa menggunakan label diagnosis. Pemilihan metode ini dilakukan untuk melihat pengelompokkan antar karakteristik, dan membandingkan hasil clustering dengan label sebenarnya untuk melihat tingkat kecocokan pola cluster. Pemilihan metode ini dilakukan karena efisien secara komputasi, cocok untuk data statistik, dan mudah diimplementasikan.

Penggunaan *Decision Tree* dan *K-Means* memberikan pendekatan yang komprehensif. *Decision Tree* sebagai prediksi dan klasifikasi, sedangkan *K-Means* sebagai eksplorasi pola pengelompokkan alami dalam data. Dengan menggunakan kedua metode ini dapat menghasilkan analisis yang akurat, dan informatif untuk memahami karakteristik data kanker payudara.

B. Kesesuaian Metode dengan Penyelesaian Masalah dalam Project

Dalam project ini mengintegrasikan dua teknik *Machine Learning* yang berbeda, yaitu algoritma *K-Means* dan *Decision Tree* untuk mencapai tujuan deteksi dan diagnosis kanker payudara yang cepat dan akurat. Algoritma *K-Means* digunakan sebagai teknik *unsupervised clustering* yang sangat relevan untuk memproses data medis. Di mana dalam konteks ini dapat digunakan untuk segmentasi atau pengelompokan piksel citra MRI guna memperjelas objek kanker, sehingga area tumor dapat diidentifikasi dan dikelompokkan dengan lebih mudah dan akurat[1]. Dengan demikian, *K-Means* berfungsi untuk eksplorasi pola pengelompokkan alami dalam data dan melihat pengelompokkan antar karakteristik fitur tumor.

Di sisi lain, algoritma *Decision Tree* dipilih sebagai teknik klasifikasi yang populer karena kemampuannya menghasilkan model transparan dan mudah dipahami untuk mendukung pengambilan keputusan medis. Metode ini digunakan untuk klasifikasi diagnosis akhir, yakni membedakan tumor jinak dan tumor ganas, sekaligus mengidentifikasi fitur yang paling menentukan hasil klasifikasi. Dalam studi sebelumnya, *Decision Tree* telah menunjukkan potensi besar dalam mengklasifikasi data kanker payudara dengan tingkat akurasi yang tinggi, yakni mencapai 95.00% dalam pengelompokan kanker jinak dan ganas[7].

Dengan menggabungkan analisis clustering menggunakan *K-Means* untuk pemrosesan data awal dan klasifikasi menggunakan *Decision Tree*, project ini bertujuan membangun sistem deteksi kanker payudara yang lebih efisien, terukur, dan memberikan interpretasi yang jelas mengenai faktor-faktor yang paling berpengaruh dalam diagnosis.

C. Penentuan Metrik Evaluasi

Dalam memilih model klasifikasi seperti *Decision Tree*, yang memiliki performa terbaik untuk sistem deteksi kanker payudara. Perlu digunakan beberapa metrik kunci dari *sklearn.metrics*, yaitu *Accuracy*, *Precision*, *Recall*, *F1-Score*, serta *Confusion Matrix*. Metrik ini sangat penting karena membantu menilai seberapa handal model dalam membedakan tumor jinak (0) dan tumor ganas (1).

1. Metrik Evaluasi Klasifikasi (*Decision Tree*)

a. *Confusion Matrix*

Confusion Matrix digunakan untuk melihat detail prediksi benar dan salah pada masing-masing kelas, yang mana matriks ini membantu mengidentifikasi kesalahan model, seperti kecenderungan memprediksi suatu kelas secara berlebihan atau kekurangan.

Keterangan;

- *True Positive (TP)*
Jumlah kasus tumor ganas yang diprediksi dengan benar sebagai ganas
- *False Positive (FP)*
Jumlah kasus tumor jinak yang salah diprediksi sebagai ganas
- *False Negative (FN)*
Jumlah kasus tumor ganas yang salah diprediksi sebagai jinak
- *True Negative (TN)*
Jumlah kasus tumor jinak yang diprediksi dengan benar sebagai jinak

b. *Accuracy*

Accuracy digunakan untuk mengukur persentase prediksi yang benar dari total data. Namun, jika hanya memakai *Accuracy* masih belum cukup, sehingga pada evaluasi ditambahkan penggunaan metrik seperti *Precision*, *Recall*, dan *F1-Score*.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

c. *Precision*

Precision menunjukkan tingkat ketepatan model dalam memprediksi kelas positif (ganas), yaitu seberapa banyak prediksi benar dibandingkan total prediksi pada kelas tersebut. *Precision* penting untuk meminimalkan *False Positive* (FP).

$$Precision = \frac{TP}{TP + FP}$$

d. *Recall*

Recall menunjukkan seberapa baik model berhasil menemukan seluruh data dalam kelas positif (ganas). *Recall* sangat penting dalam deteksi kanker payudara karena berfungsi meminimalkan *False Negative* (FN).

$$Recall = \frac{TP}{TP + FN}$$

e. *F1-Score*

F1-Score digunakan untuk menunjukkan rata-rata harmonik dari *Precision* dan *Recall* sehingga memberikan penilaian yang lebih seimbang, terutama pada kondisi data dengan distribusi kelas yang tidak merata.

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

f. *Average Metrics*

Macro Average digunakan untuk menghitung rata-rata metrik (*Precision*, *Recall*, *F1-Score*) pada setiap kelas tanpa mempertimbangkan jumlah data. Ini menunjukkan keseimbangan prediksi antar kelas. Sedangkan, *Weighted Average* digunakan untuk mempertimbangkan banyaknya data pada masing-masing kelas, sehingga memberikan gambaran yang lebih sesuai secara keseluruhan, terutama ketika jumlah data tiap kelas tidak sama.

2. Metrik Evaluasi *Clustering* (*K-Means*)

Untuk menilai efektivitas tahap clustering *K-Means* yang digunakan sebagai eksplorasi pola pengelompokan alami dalam data, metrik internal *clustering* seperti *Silhouette Score* juga perlu ditentukan. Skor ini mengukur seberapa mirip suatu titik data dengan clusternya sendiri dibandingkan dengan *cluster* terdekat lainnya untuk menilai kualitas pengelompokan.

BAB IV

HASIL DAN ANALISIS

A. Hasil Eksperimen atau Pemodelan

Pada project ini menggunakan 2 algoritma, yaitu *K-Means* untuk *clustering* dan Klasifikasi menggunakan *Decision Tree*.

1. Pre-Processing

Pada tahap ini dilakukan *Pre-Processing* pada algoritma *K-Means* dan *Decision Tree*. Tujuannya untuk melakukan standarisasi agar semua fitur berkontribusi sama dalam perhitungan jarak dan menghasilkan *cluster* yang valid pada algoritma *K-Means*. Sedangkan untuk *Decision Tree* tujuannya untuk menyiapkan variabel target agar algoritma dapat membangun aturan keputusan secara efektif.

a. Decision Tree

- Read dataset untuk dilakukan *Pre-Processing*

```
data = pd.read_csv('/content/Breast_cancer_dataset.csv')
data
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	perimeter_worst	area_worst
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	...	17.33	184.60	2019.0
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08960	0.07017	...	23.41	158.80	1956.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	...	25.53	152.50	1709.0
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	...	26.50	98.87	567.7
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	...	16.67	152.20	1575.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	...	26.40	166.10	2027.0
565	926682	M	20.13	28.25	131.20	1261.0	0.06780	0.10340	0.14400	0.06791	...	38.25	155.00	1731.0
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.06251	0.05302	...	34.12	126.70	1124.0
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	...	39.42	184.60	1821.0
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	...	30.37	59.16	268.6

569 rows x 33 columns

Gambar 7. Read Dataset

- Hapus kolom 'id' dan 'Unnamed'

```
drop_cols = ['id', 'Unnamed: 32']
data = data.drop(columns=[c for c in drop_cols if c in data.columns])
data
```

Gambar 8. Kolom Id dan Unnamed

- Mengisi nilai yang hilang (*missing values*) di setiap kolom

```
for col in data.columns:
    if data[col].dtype != 'object': # numerik
        data[col] = data[col].fillna(data[col].median())
    else: # kategorikal
        data[col] = data[col].fillna(data[col].mode()[0])
```

Gambar 9. Missing Values

- Mengubah (*encode*) nilai-nilai kategorikal pada kolom 'diagnosis' menjadi nilai numerik

```
data['diagnosis'] = data['diagnosis'].map({'M': 1, 'B': 0})
data['diagnosis']
```

Gambar 10. Encoding

Nilai 'M' (Malignant/Ganas) dipetakan menjadi 0

Nilai 'B' (Benign/Jinak) dipetakan menjadi 1

diagnosis	
0	1
1	1
2	1
3	1
4	1
...	...

Gambar 11. Hasil Encoding

- Menghitung total jumlah nilai yang hilang (*missing values*)

```
data.isnull().sum()
```

Gambar 12. Total Missing Values

Tidak ada nilai yang hilang (*missing value*), data siap untuk diolah ke tahap berikutnya.

	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0

Gambar 13. Hasil Missing Values

- Simpan data

```
data.to_csv('data decision tree.csv')
```

Gambar 14. Simpan Data Bersih

b. *K-Means*

- *Import StandardScaler* untuk normalisasi pada data untuk K-Means

```
from sklearn.preprocessing import StandardScaler
```

Gambar 15. Import Library

- Read dataset untuk dilakukan *Pre-Processing* seperti gambar 7.
- Hapus kolom 'id' dan 'Unnamed' pada gambar 8.
- Mengisi nilai yang hilang (*missing values*) di setiap kolom pada gambar 9.
- Memisahkan data menjadi variabel X (matriks fitur) dan y (kolom target 'diagnosis' yang sudah di-encode).

```
selected_features = ['radius_mean', 'texture_mean', 'perimeter_mean',
                    'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
                    'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
                    'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
                    'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
                    'fractal_dimension_se', 'radius_worst', 'texture_worst', 'perimeter_worst',
                    'area_worst', 'smoothness_worst', 'compactness_worst', 'concavity_worst',
                    'concave points_worst', 'symmetry_worst', 'fractal_dimension_worst']

X = data[selected_features]
y = data['diagnosis']
```

Gambar 16. Memisahkan data menjadi variabel

- Melakukan Standarisasi pada fitur-fitur numerik yang ada di variabel X

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Gambar 17. Standarisasi Fitur Numerik

- Menggabungkan fitur yang telah distandarisasi (X_scaled) dengan kolom target yang sudah di-encode (y)

```
X_scaled_df = pd.DataFrame(X_scaled, columns=selected_features)
X_scaled_df['diagnosis'] = y
```

Gambar 18. Penggabungan fitur yang telah distandarisasi

- Simpan data

```
X_scaled_df.to_csv('data K-Means.csv', index=False)
```

Gambar 18. Simpan Data K-Means

2. Hasil *Clustering* dengan *K-Means*

a. *Import library* yang dibutuhkan

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import pandas as pd
import matplotlib.pyplot as plt
```

Gambar 19. Import Library

b. Load data *K-Means*

```
data = pd.read_csv('/content/data K-Means.csv')
data
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean
0	1.097064	-2.073335	1.269934	0.984375	1.568466	3.283515	2.652874	2.532475	2.217515
1	1.829821	-0.353632	1.685955	1.908708	-0.826962	-0.487072	-0.023846	0.548144	0.001392
2	1.579888	0.456187	1.566503	1.558884	0.942210	1.052926	1.363478	2.037231	0.939685
3	-0.768909	0.253732	-0.592687	-0.764464	3.283553	3.402909	1.915897	1.451707	2.867383
4	1.750297	-1.151816	1.776573	1.826229	0.280372	0.539340	1.371011	1.428493	-0.009560
...	...	...	...	...	...	...	...	...	...
564	2.110995	0.721473	2.060786	2.343856	1.041842	0.219060	1.947285	2.320965	-0.312589
565	1.704854	2.085134	1.615931	1.723842	0.102458	-0.017833	0.693043	1.263669	-0.217664
566	0.702284	2.045574	0.672676	0.577953	-0.840484	-0.038680	0.046588	0.105777	-0.809117
567	1.838341	2.336457	1.982524	1.735218	1.525767	3.272144	3.296944	2.658866	2.137194
568	-1.808401	1.221792	-1.814389	-1.347789	-3.112085	-1.150752	-1.114873	-1.261820	-0.820070

569 rows × 10 columns

Gambar 20. Baca Data *K-Means*

c. Menyimpan semua fitur pada daftar features dari DataFrame 'data' ke dalam variabel X

```
features = ['radius_mean', 'texture_mean', 'perimeter_mean',
            'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
            'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
            'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
            'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
            'fractal_dimension_se', 'radius_worst', 'texture_worst', 'perimeter_worst',
            'area_worst', 'smoothness_worst', 'compactness_worst', 'concavity_worst',
            'concave points_worst', 'symmetry_worst', 'fractal_dimension_worst']

X = data[features]
```

Gambar 21. Mengambil Semua Fitur

d. Menentukan jumlah *cluster* optimal (nilai *k* terbaik) dalam analisis *clustering*

```
from sklearn.cluster import KMeans
k_values = range(2, 10)

inertia = []
sil_scores = []

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)

    inertia.append(km.inertia_)
    sil_scores.append(silhouette_score(X, km.labels_))
```

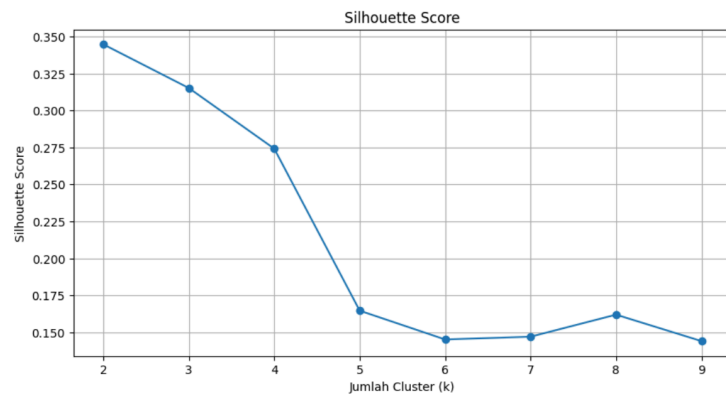
Gambar 22. Menentukan Jumlah Cluster

- e. Memvisualisasikan hasil perhitungan *Silhouette Score* dari berbagai jumlah *cluster* (*k*)

```
plt.figure(figsize=(10,5))
plt.plot(k_values, sil_scores, marker='o')
plt.title('Silhouette Score')
plt.xlabel('Jumlah Cluster (k)')
plt.ylabel('Silhouette Score')
plt.grid()
plt.show()
```

Gambar 23. Visualisasi Silhouette Score

Grafik menunjukkan jumlah *cluster* (*k*) yang optimal untuk *clustering* data adalah 2



Gambar 24. Grafik Jumlah Cluster

- f. *Clustering K-Means* pada data X

```
kmeans = KMeans(n_clusters=2, random_state=42)
clusters = kmeans.fit_predict(X)
```

Gambar 25. Clustering K-Means

- g. Menambahkan hasil *clustering* sebagai kolom baru pada data

```
data['cluster'] = clusters
data
```

...	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_worst	diagnosis	cluster
...	2.303601	2.001237	1.307686	2.616665	2.109526	2.296076	2.750622	1.937015	M	0
...	1.535126	1.890489	-0.375612	-0.430444	-0.146749	1.087084	-0.243890	0.281190	M	0
...	1.347475	1.456285	0.527407	1.082932	0.854974	1.955000	1.152255	0.201391	M	0
...	-0.249939	-0.550021	3.394275	3.893397	1.989588	2.175786	6.046041	4.935010	M	0
...	1.338539	1.220724	0.220556	-0.313395	0.613179	0.729259	-0.868353	-0.397100	M	0
...	...	...	...	...	...	...	...	...	...	...
...	1.752563	2.015301	0.378365	-0.273318	0.664512	1.629151	-1.360158	-0.709091	M	0
...	1.421940	1.494959	-0.691230	-0.394820	0.236573	0.733827	-0.531855	-0.973978	M	0
...	0.579001	0.427906	-0.809587	0.350735	0.326767	0.414069	-1.104549	-0.318409	M	0
...	2.303601	1.653171	1.430427	3.904848	3.197605	2.289885	1.919083	2.219635	M	0
...	-1.432735	-1.075813	-1.859019	-1.207552	-1.305831	-1.745063	-0.048138	-0.751207	B	1

Gambar 26. Kolom Cluster

- h. Menghitung jumlah baris data di setiap *cluster*

```
data['cluster'].value_counts()
```

count	
cluster	
1	381
0	188

Gambar 27. Jumlah Data Setiap Cluster

- i. Menampilkan data pada *cluster* 0 (*Malignant/ Ganas*)

```
data[data['cluster'] == 0]
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean
0	1.097064	-2.073335	1.269934	0.984375	1.568466	3.283515	2.652874	2.532475	2.217515
1	1.829821	-0.353632	1.685955	1.908708	-0.826962	-0.487072	-0.023846	0.548144	0.001392
2	1.579888	0.456187	1.566503	1.558884	0.942210	1.052926	1.363478	2.037231	0.939685
3	-0.768909	0.253732	-0.592687	-0.764464	3.283553	3.402909	1.915897	1.451707	2.867383
4	1.750297	-1.151816	1.776573	1.826229	0.280372	0.539340	1.371011	1.428493	-0.009560
...	...	...	...	...	...	...	...	...	...
563	1.929226	1.349781	2.101976	1.968434	0.963560	2.260135	2.870075	2.540213	1.231760
564	2.110995	0.721473	2.060786	2.343856	1.041842	0.219060	1.947285	2.320965	-0.312589
565	1.704854	2.085134	1.615931	1.723842	0.102458	-0.017833	0.693043	1.263669	-0.217664
566	0.702284	2.045574	0.672676	0.577953	-0.840484	-0.038680	0.046588	0.105777	-0.809117
567	1.838341	2.336457	1.982524	1.735218	1.525767	3.272144	3.296944	2.658866	2.137194

188 rows × 32 columns

Gambar 28. Menampilkan Data Cluster

- j. Menampilkan data pada *cluster* 1 (*Benign/ Jinak*)

```
data[data['cluster'] == 1]
```

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean
10	0.537556	0.919273	0.442011	0.406453	-1.017686	-0.713542	-0.700684	-0.404686	-1.035476
13	0.489274	1.084495	0.483201	0.363507	-0.878913	-0.078478	0.132840	0.121770	0.129175
16	0.156977	0.195555	0.114137	0.084216	0.164372	-0.612909	-0.186433	0.094686	-0.823721
19	-0.166799	-1.147162	-0.185728	-0.251957	0.101747	-0.436850	-0.278210	-0.028609	0.267911
20	-0.297446	-0.833008	-0.261106	-0.383638	0.792763	0.429422	-0.541362	-0.459627	0.567289
...	...	...	...	...	...	...	...	...	...
558	0.131416	0.788958	0.182100	0.006288	-0.827674	0.543131	0.177034	-0.298156	-1.305645
559	-0.743348	1.079841	-0.718729	-0.714976	-0.266890	-0.042470	0.281240	-0.202977	-1.546608
560	-0.021952	1.829157	-0.024262	-0.154973	0.208495	0.156521	-0.554670	-0.151647	-1.002617
561	-0.831392	2.345765	-0.877311	-0.764748	-1.556408	-1.303122	-1.114873	-1.261820	-2.744117
568	-1.808401	1.221792	-1.814389	-1.347789	-3.112085	-1.150752	-1.114873	-1.261820	-0.820070

381 rows × 32 columns

Gambar 29. Menampilkan Data Cluster 1

- k. Menghitung jarak *Euclidean* antara dua *centroid* (titik pusat) dari *Cluster 0* dan *Cluster 1*

```
centroids = kmeans.cluster_centers_  
for i in range(len(centroids)):  
    for j in range(i+1, len(centroids)):  
        dist = np.linalg.norm(centroids[i] - centroids[j])  
        print(f"Jarak Cluster {i} ke Cluster {j}: {dist}")
```

Jarak Cluster 0 ke Cluster 1: 6.594466699517434

Gambar 30. Menghitung Jarak Euclidean antara Dua Centroid

- l. Menampilkan koordinat dari dua *centroid* (2 *Cluster*)

```
print(kmeans.cluster_centers_)  
  
[[ 0.98649786  0.49202131  1.01866689  0.97479724  0.5871176  1.014073  
 1.14492245  1.17028266  0.60339021  0.22927434  0.86311672  0.04416341  
 0.86446528  0.8137762  0.01228944  0.69281919  0.63976499  0.77166695  
 0.13798752  0.40384985  1.05221312  0.51705679  1.07769473  1.01391704  
 0.59804381  0.95285513  1.05144274  1.15328841  0.5994129  0.61362004]  
 [-0.48677585 -0.24278217 -0.50264928 -0.48100231 -0.28970632 -0.50038248  
 -0.56494861 -0.57746231 -0.29773585 -0.11313275 -0.42589487 -0.02179192  
 -0.4265603  -0.40154836 -0.00606408 -0.34186354 -0.31568456 -0.38077004  
 -0.06808833 -0.19927499 -0.51920227 -0.25513563 -0.53177588 -0.50030552  
 -0.29509774 -0.47017523 -0.51882214 -0.56907669 -0.29577329 -0.30278364]]
```

Gambar 31. Menampilkan Koordinat dari Dua Centroid

- m. Data berhasil dibagi menjadi dua *cluster* yang terpisah dengan baik, dan kedua *cluster* menunjukkan karakteristik yang berbeda secara signifikan.

```
score = silhouette_score(X, kmeans.labels_)  
print(f"k = {2}, Silhouette Score = {score}")
```

k = 2, Silhouette Score = 0.34473443466110537

Gambar 32. Data Berhasil menjadi Dua Cluster

- n. Simpan data

```
data.to_csv('data cluster K-Means.csv')
```

Gambar 33. Data Cluster K-Means

Visualisasi K-Means

- a. Reduksi dimensi pada data menggunakan *Principal Component Analysis* (PCA), mengurangi dimensi fitur menjadi dua komponen utama PC1 dan PC2.

	PC1	PC2	cluster
0	9.192837	1.948583	0
1	2.387802	-3.768172	0
2	5.733896	-1.075174	0
3	7.122953	10.275589	0
4	3.935302	-1.948072	0
...	...	...	...
564	6.439315	-3.576817	0
565	3.793382	-3.584048	0
566	1.256179	-1.902297	0
567	10.374794	1.672010	0
568	-5.475243	-0.670637	1

569 rows × 3 columns

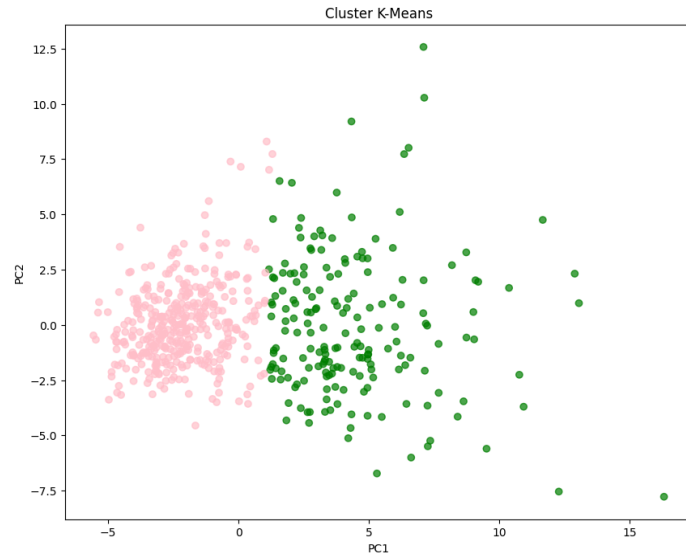
Gambar 34. Reduksi Dimensi

- b. Visualisasi plot

```
import numpy as np
import matplotlib.pyplot as plt
colors = np.where(pca_df['cluster'] == 0, 'green', 'pink')
#green untuk cluster 0 dan pink untuk cluster 1
plt.figure(figsize=(10,8))
plt.scatter(
    pca_df['PC1'],
    pca_df['PC2'],
    c=colors,
    alpha=0.7
)

plt.title('Cluster K-Means')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

Gambar 35. Visualisasi Reduksi



Gambar 36. Hasil Visualisasi Reduksi

Grafik plot tersebut menunjukkan hasil *clustering K-Means* pada data yang telah direduksi dimensinya menggunakan PCA. Sumbu X menandakan PC1 dan sumbu Y adalah PC2, kedua sumbu ini mewakili dua komponen utama yang menangkap varian terbesar dari data asli. Terdapat dua *cluster* yang ditunjukkan oleh warna hijau untuk *cluster* 0 dan merah muda untuk *cluster* 1.

Ada sedikit tumpang tindih di sekitar area dimana PC1 bernilai antara -1 dan 2, menunjukkan adanya beberapa titik data yang memiliki karakteristik fitur yang mirip dan bisa saja masuk ke salah satu kelompok, namun algoritma *K-Means* mengklasifikasikannya berdasarkan kedekatan *centroid*. Plot ini secara visual validasi bahwa data dapat dikelompokkan menjadi dua segmen berbeda berdasarkan fitur-fitur yang direpresentasikan oleh dua komponen utama (PC1 dan PC2)

c. Visualisasi hasil *clustering K-Means* dengan *centroid* (titik pusat)

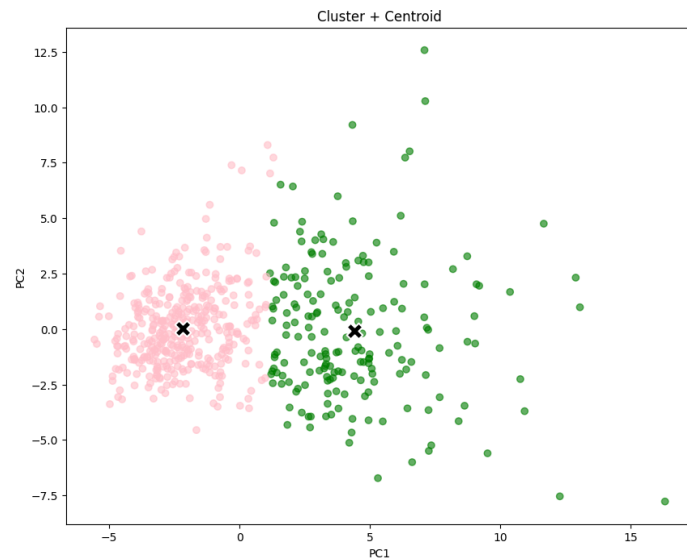
```
from sklearn.decomposition import PCA
pca_for_centroids = PCA(n_components=2)
X_for_pca_fit = data[features]
pca_for_centroids.fit(X_for_pca_fit)

centroids = kmeans.cluster_centers_
centroids_pca = pca_for_centroids.transform(centroids)

colors = np.where(pca_df['cluster'] == 0, 'green', 'pink')

plt.figure(figsize=(10,8))
plt.scatter(pca_df['PC1'], pca_df['PC2'], c=colors, alpha=0.6)
plt.scatter(centroids_pca[:,0], centroids_pca[:,1], s=200, c='black', marker='x', edgecolor='white', linewidth=1.5)
plt.title('Cluster + Centroid')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

Gambar 37. Visualisasi Centroid



Gambar 38. Hasil Visualisasi Centroid

Plot tersebut menunjukkan lokasi *centroid* dari masing-masing *cluster*. tanda 'x' pada *cluster* 1 (merah muda) merupakan titik pusat rata-rata dari semua titik data. Lokasinya sangat dekat dengan massa utama *cluster* 1(merah muda) menunjukkan bahwa *cluster* tersebut padat dan terpusat dengan baik. *Centroid* pada *cluster* 0 (hijau) tampak lebih menyebar, variasinya lebih besar dan *centroid* ini berada di pusat massa yang luas.

d. Plot *Benign vs. Malignant*

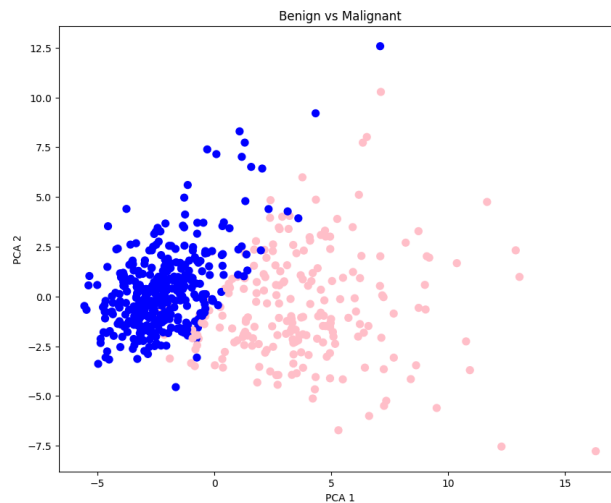
```
y_diagnosis = data['diagnosis']
colors = y_diagnosis.map({'B':'blue', 'M':'pink'})

plt.figure(figsize=(10,8))
plt.scatter(
    X_pca[:,0],
    X_pca[:,1],
    c=colors,
    s=50
)

plt.title("Benign vs Malignant")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.show()
```

Gambar 38. Visualisasi plot M vs. B

Membandingkan secara visual hasil pengelompokan *K-Means* antara *Benign* dan *Malignant*.



Gambar 39. Hasil visualisasi plot *M vs. B*

3. Hasil Klasifikasi dengan *Decision Tree*

a. *Import library* yang dibutuhkan

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
import sklearn.tree as tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
```

Gambar 40. *Import Library*

b. Load data *Decision Tree*

```
data = pd.read_csv('/content/data decision tree.csv')
data
```

	Unnamed: 0	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...
0	0	1	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	...
1	1	1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	...
2	2	1	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	...
3	3	1	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	...
4	4	1	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	...
...	...	...	...	...	...	...	...	...	...	...	...
564	564	1	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	...
565	565	1	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	...
566	566	1	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	...
567	567	1	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	...
568	568	0	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	...

Gambar 41. Baca Dataset *Decision Tree*

c. Menghapus kolom '*Unnamed*' seperti pada gambar 8.

d. Membagi dataset

Dataset dibagi menjadi 2, data pelatihan (X_{train} , y_{train}) dan data uji

(X_{test} , y_{test}), dimana 30% dari data asli untuk pengujian ($test_size=0.3$) dan sisanya (70%) untuk pelatihan.

```
# Perform the train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

Gambar 42. Membagi Dataset

- e. Penyetelan *hyperparameter* otomatis pada model *Decision Tree Classifier* menggunakan *Grid Search Cross-Validation* (*GridSearchCV*).

```
dt = DecisionTreeClassifier(random_state=42)

param_grid = {'max_depth': np.arange(1, 6)}

grid = GridSearchCV(dt, param_grid=param_grid, cv=5)

grid.fit(X_train, y_train)

print("Parameter terbaik:", grid.best_params_)
print("Akurasi Cross Validation terbaik:", grid.best_score_)
```

Gambar 43. Setel Hyperparameter

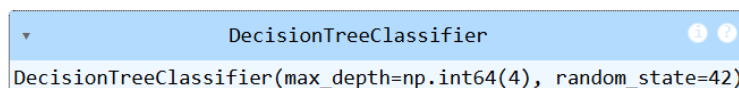
max_depth , dari semua kedalaman *decision tree* (1 hingga 5) yang diuji, kedalaman 4 menghasilkan kinerja rata-rata terbaik. Dan akurasi rata-rata tertinggi yang dicapai oleh model dengan $max_depth=4$ selama proses sekitar 92.72%.

```
Parameter terbaik: {'max_depth': np.int64(4)}
Akurasi Cross Validation terbaik: 0.9271835443037976
```

Gambar 44. Parameter Terbaik

- f. Mengambil model *Decision Tree* dengan *hyperparameter* terbaik ($max_depth=4$) dan melatihnya kembali pada seluruh data pelatihan.

```
best_dt = grid.best_estimator_
best_dt.fit(X_train, y_train)
```



Gambar 45. Model Decision Tree

- g. Penyetelan *hyperparameter*, pelatihan model terbaik, dan prediksi pada data baru.

```
new_data = [X.mean().values]
prediction = best_dt.predict(new_data)
print("Prediction:", prediction)
prediction_label = "Malignant" if prediction[0] == 1 else "Benign"
print("Prediction Label:", prediction_label)
```

Gambar 46. Setel Hyperparameter

Model *Decision Tree* yang telah dioptimalkan memprediksi bahwa data *input* baru yang nilainya merupakan rata-rata dari semua fitur dalam dataset tergolong sebagai *Benign* (Jinak).

```
Prediction: ['B']
Prediction Label: Benign
```

Gambar 47. Rata-Rata Prediksi

- h. Menguji kinerja akhir model

```
y_pred = best_dt.predict(X_test)

print("\nAkurasi Test:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Gambar 48. Uji Kinerja Model

Model *Decision Tree* mencapai akurasi sebesar 95.32% pada data pengujian. Ini berarti bahwa model berhasil memprediksi label (*Benign* atau *Malignant*) dengan benar untuk lebih dari 95% sampel dalam set pengujian.

Akurasi Test: 0.9532163742690059

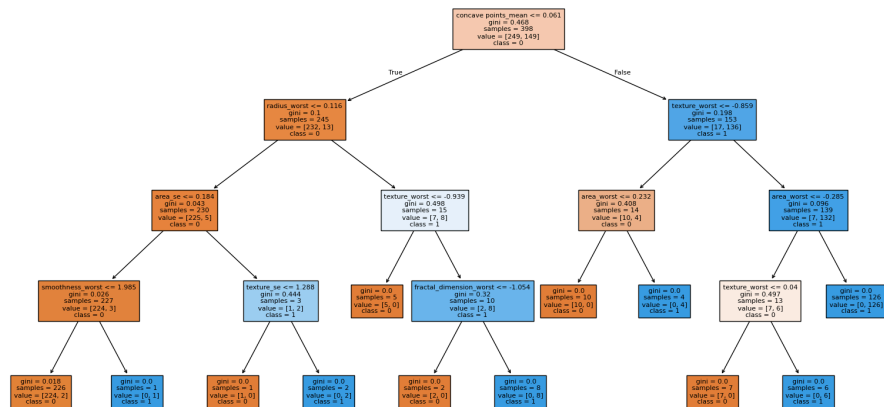
Classification Report:				
	precision	recall	f1-score	support
B	0.96	0.96	0.96	108
M	0.94	0.94	0.94	63
accuracy			0.95	171
macro avg			0.95	171
weighted avg			0.95	171

Gambar 49. Akurasi dan Klasifikasi

i. Visualisasi struktur dari model *Decision Tree*

```
class_names = ['0', '1']
plt.figure(figsize=(20,10))
plot_tree(
    best_dt,
    feature_names=X.columns,
    class_names=class_names,
    filled=True,
    fontsize=8
)
plt.show()
```

Gambar 50. Visualisasi *Decision Tree*



Gambar 51. Hasil *Decision Tree*

B. Analisis atau Penjelasan Hasil Eksperimen

1. *K-Means Clustering*

Analisis ini bertujuan untuk menemukan *cluster* alami dalam data fitur (X) tanpa menggunakan label kelas yang ada.

a. Membersihkan data dan menyiapkan fitur

Sebelum tahap analisis lebih lanjut, data dibersihkan dengan menghapus kolom-kolom yang tidak relevan, seperti kolom 'Unnamed'. Tahap ini memastikan hanya fitur-fitur penting yang digunakan terdiri dari 30 fitur yang telah dinormalisasi.

b. Menentukan jumlah *cluster* (*k*) optimal

Pada project ini menggunakan metrik Silhouette Score, untuk menentukan jumlah *cluster* (*k*) yang tepat. Grafik *Silhouette Score* menunjukkan puncak tertinggi pada nilai *k* = 2, skor tertinggi di angka 0,3447. Karena skor tersebut jauh lebih tinggi daripada skor untuk *k* lainnya, maka *k* = 2 ditetapkan sebagai jumlah *cluster* yang optimal.

c. Hasil *Clustering*

Model *K-Means* kemudian diinisialisasi dan dilatih menggunakan *k* = 2. Setelah itu, setiap titik data diberi label *cluster* 0 atau 1, dan ditambahkan ke dalam DataFrame. Perhitungan frekuensi

menunjukkan pembagian, *cluster* 1 dengan 381 baris data dan pada *cluster* 0 sebanyak 188 baris data.

d. Karakterisasi *Cluster*

Perhitungan jarak *cluster* antara dua *centroid* dilakukan dengan menggunakan *Euclidean*. Dan hasilnya jarak antara *cluster* 0 ke *cluster* 1 adalah 6,594. Jarak yang signifikan ini mendukung temuan *Silhouette Score*, yang mengindikasikan bahwa kedua *cluster* tersebut terpisah dengan baik.

Analisis *centroid* (titik pusat), *cluster* 0 memiliki nilai *centroid* yang didominasi oleh bilangan positif (misalnya, *radius_mean* = 0,98). Dan pada *cluster* 1 memiliki nilai *centroid* yang didominasi oleh bilangan negatif (misalnya, *radius_mean* = -0,48).

Hasil analisis menunjukkan bahwa algoritma *K-Means* berhasil mengelompokkan data menjadi dua kelompok yang sangat berbeda berdasarkan fitur-fitur yang tersedia. Pilihan *cluster k* = 2 didukung oleh *Silhouette Score* yang tinggi dan jarak antar *centroid* yang besar, menunjukkan pemisahan alami menjadi dua kelompok dalam data.

Cluster 0 mewakili baris data (188 data) dengan nilai-nilai fitur yang lebih tinggi dari rata-rata populasi. Sedangkan *cluster* 1 mewakili baris data (381 data) dengan nilai-nilai fitur yang lebih rendah dari rata-rata populasi. Perbedaan nilai fitur yang ekstrem seringkali berkorelasi kuat dengan kelas diagnosis yang berbeda. Misalnya, *cluster* 0 kemungkinan mewakili kasus *Malignant* (ganas), sementara *cluster* 1 mewakili kasus *Benign* (jinak).

2. *Decision Tree Classifier*

Analisis ini bertujuan untuk melatih model *Decision Tree* dalam mengklasifikasikan data menggunakan teknik *hyperparameter tuning* dan evaluasi yang ketat.

a. *Optimalisasi Hyperparameter*

Tahap awal adalah mencari kedalaman *Decision Tree* (*max_depth*) yang optimal menggunakan *Grid Search Cross-Validation (CV) 5-fold* dapat dilihat dalam gambar 42. Output menunjukkan *max_depth* = 4, kedalaman 4 memberikan kinerja generalisasi terbaik. Akurasi CV terbaik = 0,9272, model dengan *max_depth* = 4 mencapai akurasi rata-rata 92,72% selama proses validasi silang pada data pelatihan.

b. Pelatihan

Model dengan *max_depth* = 4 kemudian diambil dan dilatih menggunakan seluruh data pelatihan untuk persiapan pengujian. visualisasi *Decision Tree* berfungsi untuk memetakan aturan keputusan yang telah dipelajari model. *Decision Tree* ini akan menunjukkan urutan fitur mana yang paling penting, berada di node paling atas dan ambang batas nilai apa yang digunakan untuk membagi data.

c. Evaluasi pada data uji

Model yang telah dioptimalkan diuji pada data pengujian (X_{test}) yang baru dalam gambar 47. Model mencapai akurasi 0,9532 atau 95,32% pada data pengujian. Kenaikan akurasi dari akurasi CV yaitu 92,72% ke akurasi test 95,32% menunjukkan bahwa model dilatih pada data set pelatihan penuh bekerja sedikit lebih baik, dan model memiliki kemampuan generalisasi yang sangat baik. Tidak *overfitting* secara signifikan, karena akurasi pengujian tetap tinggi.

Nilai *Precision*, *Recall*, dan *F1-Score* yang sangat tinggi dan seimbang untuk kedua kelas. *Recall* 0,94 menunjukkan model berhasil mendeteksi 94% dari semua kasus *Malignant* yang sebenarnya ada. Dan *Precision* 0,94 menunjukkan ketika model prediksi kasus sebagai *Malignant*. Kinerja yang seimbang ini menunjukkan bahwa model sangat handal dan memiliki bias yang minimal terhadap salah satu kelas.

Analisis ini menghasilkan model *Decision Tree* yang optimal dengan akurasi pengujian yang tinggi yaitu 95,32% dan metrik kinerja yang seimbang untuk kedua kelas. Model tersebut menunjukkan kinerja yang stabil dan handal, membuatnya efektif untuk memprediksi label kelas pada data baru.

C. Interactive Search

```
tree_ = best_dt.tree_
feature_names = X.columns.tolist()

def follow_tree():
    print("INTERACTIVE DECISION TREE SEARCH")
    print("Masukkan nilai fitur yang akan digunakan untuk traversal Tree.\n")

    # Input nilai semua fitur
    values = {}
    for col in feature_names:
        values[col] = float(input(f"{col}: "))

    print("HASIL PENELUSURAN TREE")

    node = 0 # mulai dari root
    while tree_.feature[node] != -2: # -2 = leaf node
        feature_index = tree_.feature[node]
        threshold = tree_.threshold[node]
        feature_name = feature_names[feature_index]
        user_value = values[feature_name]

        print(f"\nNode cek: {feature_name} <= {threshold:.3f}?\n")
        print(f"Nilai input kamu: {user_value}")

        if user_value <= threshold:
            print("→ Ke cabang: TRUE")
            node = tree_.children_left[node]
        else:
            print("→ Ke cabang: FALSE")
            node = tree_.children_right[node]

    # keluar loop → leaf node
    prediction = np.argmax(tree_.value[node])
    label = "Malignant (ganas)" if prediction == 1 else "Benign(jinak)"

    print("LEAF NODE DICAPAI")
    print("Prediksi model:", label)
    print("Distribusi kelas di leaf:", tree_.value[node])

# Panggil fungsi
follow_tree()
```

Gambar 52. Interactive Search

Fungsi *code* tersebut untuk melakukan *Interactive Search* pada *Decision Tree*. *Interactive Search* meminta pengguna untuk memasukkan nilai pada setiap fitur yang digunakan oleh model. Kemudian, model akan melacak jalur dari *root* hingga ke *leaf node* dalam *Decision Tree*, membandingkan nilai *input* pengguna dengan ambang batas (*threshold*) pada setiap simpul *internal*.

Setelah mencapai *leaf node*, fungsi akan memberikan prediksi berdasarkan distribusi kelas di simpul tersebut, lalu mencetak label prediksi antara ‘*Malignant*’ (ganas) atau ‘*Benign*’ (jinak) dan distribusi kelas yang dicapai.

```
INTERACTIVE DECISION TREE SEARCH
Masukkan nilai fitur yang akan digunakan untuk traversal Tree.

radius_mean: 1.647
texture_mean: 0.792
perimeter_mean: 2.380
area_mean: 0.978
smoothness_mean: 1.3863
compactness_mean: 0.9725
concavity_mean: 0.8653
concave points_mean: 0.0976
symmetry_mean: 1.3647
fractal_dimension_mean: 0.0791
radius_se: 1.0373
texture_se: 0.8642
perimeter_se: 1.086
area_se: 0.07542
smoothness_se: 1.378
compactness_se: 0.086
concavity_se: 1.03275
concave points_se: 0.9870
symmetry_se: 1.3495
fractal_dimension_se: 0.9771
radius_worst: 1.3546
texture_worst: 0.0875
perimeter_worst: 1.4683
area_worst: 0.386
smoothness_worst: 1.372
compactness_worst: 0.1972
concavity_worst: 0.4567
concave points_worst: 1.2098
symmetry_worst: 0.8676
fractal_dimension_worst: 1.345
HASIL PENELUSURAN TREE

Node cek: concave points_mean <= 0.061?
Nilai input kamu: 0.0976
→ Ke cabang: FALSE

Node cek: texture_worst <= -0.859?
Nilai input kamu: 0.0875
→ Ke cabang: FALSE

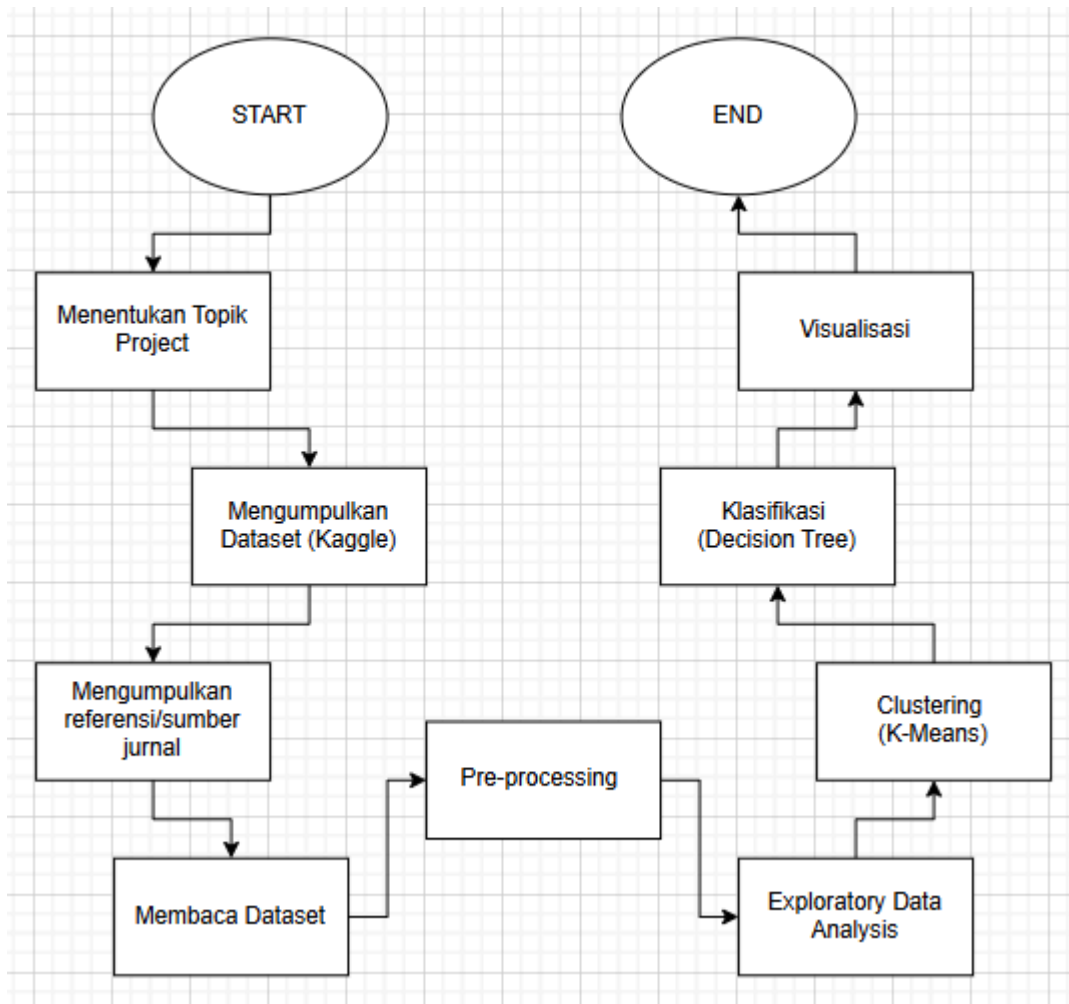
Node cek: area_worst <= -0.285?
Nilai input kamu: 0.386
→ Ke cabang: FALSE
LEAF NODE DICAPAI
Prediksi model: Malignant (ganas)
Distribusi kelas di leaf: [[0. 1.]]
```

Gambar 53. Output Interactive Search

Penelusuran dimulai dengan nilai-nilai fitur yang dimasukkan oleh pengguna. Prediksi model pada contoh penelusuran gambar diatas menunjukkan hasil *Malignant*

(ganas), dengan distribusi di leaf $[[0.1.]]$. Angka tersebut menunjukkan bahwa di *leaf node* terakhir, prediksi untuk kelas *Malignant* adalah 1, dan untuk kelas *Benign* adalah 0, sehingga prediksi akhirnya adalah *Malignant* (ganas).

D. Diagram Alur



Gambar 54. Diagram Alur

BAB V

KESIMPULAN DAN SARAN

A. Kesimpulan

Project ini bertujuan untuk membangun sistem deteksi kanker payudara yang cepat, akurat, dan interpretatif melalui integrasi teknik *Machine Learning*, yaitu algoritma *K-Means Clustering* dan *Decision Tree Classifier*. Berdasarkan hasil eksperimen dan analisis, dapat disimpulkan bahwa tujuan project telah tercapai dengan efektif.

Pada tahap eksplorasi data awal, algoritma *K-Means* berhasil mengidentifikasi struktur alami dalam data. Penentuan jumlah *cluster* optimal ($k=2$) divalidasi oleh *Silhouette Score* tertinggi (0,3447), terindikasi bahwa data fitur secara inheren terbagi menjadi dua kelompok yang terpisah dengan baik, dan dikonfirmasi oleh jarak antar *centroid* yang signifikan (6,594). **Cluster 0** sebagai nilai fitur yang lebih tinggi (kemungkinan terkait tumor ganas/*Malignant*), sedangkan **Cluster 1** sebagai nilai fitur yang lebih rendah (kemungkinan terkait tumor jinak/*Benign*). Hasil *clustering* ini memberikan dasar eksplorasi yang kuat dan mendukung pemisahan kelas dalam data.

Pada tahap klasifikasi, algoritma *Decision Tree* terbukti sangat efektif. Melalui proses *Grid Search Cross-Validation*, model dioptimalkan pada kedalaman terbaik ($max_depth = 4$), menghasilkan kinerja generalisasi yang stabil. Pada pengujian data baru, model mencapai akurasi sebesar 95,32%. Kinerja ini didukung oleh metrik lain yang seimbang dan tinggi, seperti *Recall* sebesar 0,94 untuk kelas *Malignant*. Keseimbangan metrik ini menegaskan bahwa model sangat cocok dalam meminimalkan *False Negative* yang krusial dalam diagnosis kanker sekaligus mempertahankan *Precision* yang tinggi.

Keunggulan *Decision Tree* yaitu sifatnya yang *transparan* dan *interpretatif*, memungkinkan identifikasi fitur-fitur yang paling berpengaruh dalam diagnosis, yang merupakan nilai tambah signifikan dalam mendukung pengambilan keputusan klinis yang objektif dan terukur. Secara keseluruhan, integrasi *K-Means* dan *Decision Tree* berhasil menciptakan sistem deteksi yang efektif, memenuhi tuntutan diagnosis medis akan kecepatan, akurasi, dan kemampuan interpretasi model.

B. Kendala Berdasarkan Hasil Eksperimen

Deteksi kanker payudara mencapai tingkat akurasi yang tinggi dan memberikan hasil yang memuaskan. Implementasi project ini terdapat beberapa tantangan yang harus dihadapi. Tantangan ini berdampak pada efisien waktu:

1. Keterbatasan Efisiensi Komputasi

Keputusan untuk menggunakan keseluruhan 30 fitur data dalam model, terutama pada tahap *clustering K-Means* dan pelatihan *Decision Tree*, menimbulkan kendala pada efisiensi komputasi. Penggunaan dimensi fitur yang tinggi ini mengakibatkan waktu eksekusi (*run time*) kode yang relatif lama, yang secara langsung mempengaruhi kecepatan *iterasi* dan proses pengujian model selama tahap optimasi.

2. Peningkatan Beban Kerja Analisis

Tantangan metodologis menuntut dilakukannya dua kali pengerjaan dan analisis hasil secara terpisah. Pengerjaan ganda ini melibatkan perbandingan kinerja model antara penggunaan 6 fitur terpilih (jika ada studi pendahuluan) dan penggunaan 30 fitur lengkap. Meskipun perbandingan ini krusial untuk memvalidasi signifikansi kontribusi seluruh fitur terhadap akurasi model, proses ini secara substansial menggandakan upaya dan waktu yang diperlukan untuk pengujian eksperimen dan penyusunan interpretasi hasil.

3. Masalah Dalam Pra-Pemrosesan yang Berulang

Kendala teknis yang bersifat repetitif adalah kebutuhan untuk secara konsisten menghapus kolom '*Unnamed*' setiap kali file data dibaca dan diolah. Kolom ini muncul sebagai artifact sisa dari proses penyimpanan data, dan meskipun penghapusannya merupakan langkah *Pre-Processing* yang sederhana, langkah manual yang berulang ini harus selalu diverifikasi keberadaannya untuk menjamin integritas dataset dan menghindari kegagalan eksekusi pada *script* pelatihan model.

C. Rencana Lanjut dari Hasil Eksperimen

Rencana lanjutan ini dirumuskan berdasarkan potensi pengembangan sistem deteksi kanker payudara yang telah divalidasi dengan performa tinggi (Accuracy=95.32\%) menggunakan integrasi K-Means dan Decision Tree, serta untuk mengatasi keterbatasan yang dihadapi selama implementasi:

1. Membangun sebuah sistem deteksi kanker payudara yang lebih efisien dan terukur.
2. Memberikan interpretasi yang jelas mengenai faktor-faktor yang paling berpengaruh dalam diagnosis.
3. Melakukan penerapan algoritma khusus prediksi, pengenalan pola, dan klasifikasi data, sehingga dapat digunakan untuk mengklasifikasikan jenis kanker payudara secara otomatis dan cepat.

DAFTAR PUSTAKA

- [1] N. R. Muntiari dan K. H. Hanif, "Klasifikasi Penyakit Kanker Payudara Menggunakan Perbandingan Algoritma Machine Learning," *J. Ilmu Komput. Dan Teknol.*, vol. 3, no. 1, hlm. 1–6, Mei 2022, doi: 10.35960/ikomti.v3i1.766.
- [2] "jurnal+Rosita+Siap+Terbit."
- [3] G. Gunawan, R. A. Zuama, R. S. Rohman, H. Sulaiman, dan M. A. Ghani, "DETEKSI OTOMATIS KANKER PAYUDARA MENGGUNAKAN METODE MORPHOLOGICAL RECONSTRUCTION DENGAN K-MEANS CLUSTERING PADA CITRA MRI," *Swabumi*, vol. 7, no. 1, hlm. 67–71, Mar 2019, doi: 10.31294/swabumi.v7i1.5662.
- [4] "jurnal+Rosita+Siap+Terbit."
- [5] "what-is-exploratory-data-analysis[1]."
- [6] "Unsupervised Learning."
- [7] I. Setiawan dan E. K. Nurnawati, "Penerapan Algoritma Decision Tree Untuk Memprediksi Kanker Payudara menggunakan Data Mining dan Machine Learning," vol. 11, no. 2, 2022.

LAMPIRAN

Link Github : [Publishing KA Kelompok 08](#)

Link YouTube: <https://youtu.be/BD2wFl9vnHI>